NPC System Design Document

Color Island — Living World AI

Overview

The game features up to **2,250 NPCs** living in a simulated world driven by a **needs-based utility system**. NPCs are not scripted on fixed schedules — they make decisions dynamically based on their current needs, personality, age, gender, and relationships. The player acts as an active participant in the world, able to help NPCs fulfill their needs.

---

1. Needs System

Every NPC has the following needs, each ranging from **-100 to 100**, where:

- `100` = need fully met
- `-100` = need critically unmet, must be addressed immediately

Needs are prioritized in tiers (same tier = roughly equal priority):

```
Tier 1 (Most Critical):   Health, Thirst, Hunger, Energy
Tier 2:                   Shelter, Hygiene
Tier 3:                   Social
Tier 4:                   Productivity
Tier 5 (Least Critical):  Leisure, Creativity, Athletics, Adventure, Knowledge
```

Need Depletion

- Each need depletes over time at its own rate
- Depletion rate is influenced by the NPC's traits (personality, age, gender)
- Some needs are time-of-day and time-of-year dependent (e.g. Productivity varies by season and job)
- At night, Energy depletes fast enough that NPCs will always prioritize sleep

---

2. NPC Traits

Each NPC has the following traits that influence behavior:

Trait	Values
Personality	Blue, Purple, Brown, Green, Yellow
Age	Toddler, Teen, Young Adult, Adult, Elder
Gender	Male, Female

These traits feed directly into the **weight function**, causing different NPCs to prioritize different needs even in identical situations.

---

3. Utility System

Weight Function

Each need i has a weight w_i that grows as the need depletes:

$$w_i = f(N_i,\ \text{traits})$$

The weight is influenced by:

- Current need level N_i
- NPC's personality, age, and gender (which bias weights up or down per need)

Action Scoring (Regular Utility)

At decision time, each NPC scores available actions:

$$a^* = \arg\max_{a \in A} \sum_i w_i \cdot U_i(a)$$

Where $U_i(a)$ is how much action a fulfills need i.

Abstraction Layers

The utility system operates at **three levels of abstraction**:

```
Macro Level    → NPC picks a ZONE to fulfill a need
                 (e.g. "I need food → go to Food District")

Zone Level     → NPC picks a PLACE within the zone
                 (e.g. "go to the bakery")

Object Level   → NPC interacts with a specific OBJECT
                 (e.g. "eat the bread on the counter")
```

This keeps pathfinding and object scanning cheap — NPCs only scan objects within their current zone/radius.

Object Utility Tables

Every object, place, and zone has a **universal lookup table** defining how much it fulfills each need. This is the same for all NPCs. Individuality comes entirely from the **weight function**, not the object definitions.

---

4. Social Utility System

The social system is **separate** from the regular utility system because social fulfillment depends on *who* is nearby, not just *what* is nearby.

Relationship Array

A **2,250 × 2,250 asymmetric 2D array** stores relationship scores between every pair of NPCs:

- Range: **-100 to 100** (negative = bad, 0 = neutral/stranger, positive = good)
- Asymmetric: NPC A's feeling toward B ≠ B's feeling toward A
- Relationships **decay over time** if NPCs don't interact

Social Decision Logic

When an NPC needs to fulfill their Social need:

1. Check relationship array for known NPCs nearby
2. Known NPCs with positive scores fulfill Social need significantly better than strangers
3. If no known NPCs are nearby → NPC may start a new friendship with a random nearby NPC

There is **no enemy/conflict system** — NPCs are not hostile to each other. Social is purely about fulfillment.

---

5. Mood System

NPCs display a **visible mood indicator** (Sims-style comic dialogue emoticons) reflecting their overall wellbeing.

- Mood is derived from the aggregate state of all needs
- Unfulfilled needs push mood negative
- Not all needs can be self-fulfilled — some require **player intervention**
- When an NPC needs help, they can approach the player with dynamic dialogue

---

6. The Player's Role

The player is an **object in the world** that can fulfill certain NPC needs. NPCs will approach the player when:

- A need cannot be self-fulfilled
- The player is the best available option nearby

This is the core gameplay loop: the player acts as a kind of roaming helper/priest figure, interacting with NPCs who need them. This is intentionally dynamic — not scripted.

---

7. Performance Architecture

Update Strategy

Rather than running the full algorithm for all 2,250 NPCs every frame, NPCs are updated in **rotation**:

- 1 NPC runs its decision algorithm per frame (or small batch per frame)
- At 60fps this means each NPC "thinks" roughly every ~37 seconds
- During off-turns, NPCs continue executing their current chosen action

Object Scanning

NPCs only scan objects within their **current zone and nearby radius** — not the entire world. A spatial grid or quadtree keeps this lookup cheap.

Pathfinding

- **Navmesh** precomputed for all maps — dramatically reduces search space
- NPCs travel to known, fixed destination types (home, job zone, market, etc.)
- Paths are only recalculated when destination changes
- Far-away or offscreen NPCs skip full pathfinding entirely

LOD System

```
Near player    → full needs algorithm, real pathfinding
Far from player → simplified updates, approximate behavior
Offscreen      → statistical position approximation only
```

---

8. Map & Residence Assignment

- Each NPC is assigned a **home zone** based on their industry/job (defined in Blueprint.png)
- Home is located near their work zone
- If an industry spans multiple zones, NPCs are evenly distributed across those zones

---

9. What This System Is NOT Doing

To keep scope manageable, the following were intentionally left out:

- **No complex memory per NPC** — NPCs don't remember specific past events
- **No schedule mode** — behavior is purely needs-driven, not timetable-driven
- **No enemy/conflict social system** — relationships only affect Social need fulfillment
- **No per-object personalization** — object utility tables are universal, not per-NPC

Summary

Component	Approach
Needs	15 needs, tiered priority, trait-influenced depletion
Decision Making	Weighted utility scoring across abstraction layers
Individuality	Weight function (personality, age, gender)
Social	Asymmetric 2D relationship array, proximity-based
Mood	Aggregate need satisfaction → emoticon display
Performance	Rotation updates + LOD + navmesh + zone-local scanning
Player Role	Dynamic object in world that can fulfill NPC needs